Going through this lab really showed me why you can't just trust a single accuracy score from one train-test split, since getting lucky with where the "easy" examples land can make a model look better than it actually is. Building the confusion matrix and plotting it as a heatmap in Task 1 made it much easier to actually see where the model was succeeding and where it was making Type I (False Positive) and Type II (False Negative) mistakes between the 'Died' and 'Survived' predictions. Running the classification report in Task 2 tied everything together by breaking down Precision, Recall, and the F1-Score, which helped me understand why you have to prioritize Recall whenever missing a real case—like a serious medical test—carries a high cost. Using 5-fold cross-validation in Task 3 also made a lot of sense practically, because splitting the data into five folds and averaging the scores gives you a much more honest look at how the model holds up across the whole dataset. On top of the evaluation metrics, I learned a good troubleshooting lesson right at the start when model.fit() threw a ValueError: in Python 3.14.7, using inplace=True on df['Age'].fillna() doesn't update the original DataFrame, so I had to rewrite it as a direct assignment (df['Age'] = df['Age'].fillna(df['Age'].median())) to actually clear out the missing values.